PROJECT – 4

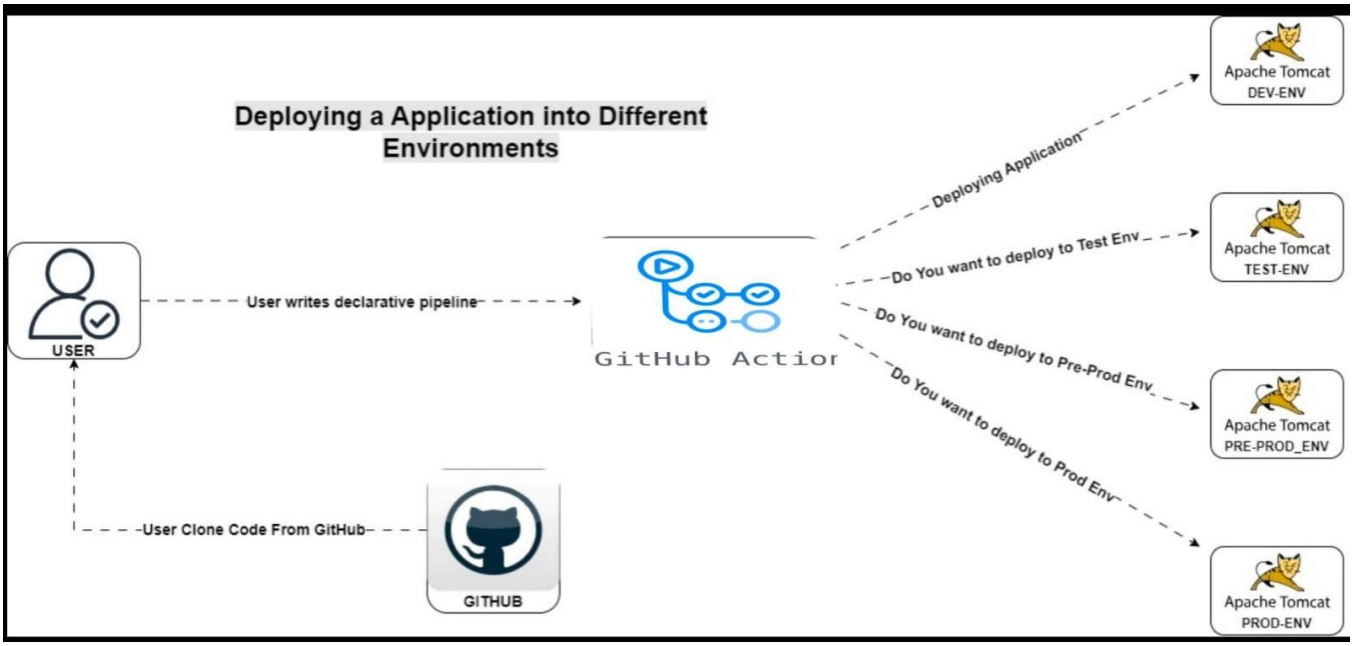
DEPLOY APPLICATION TO TOMCAT USING CI/CD PIPELINE WITH GITHUB ACTIONS

Presentation By : M.Karthik Reddy

Batch : 141

Institute : Vcube Software Solutions

DEPLOY APPLICATION TO TOMCAT USING CI/CD PIPELINE WITH GITHUB ACTIONS



Tools Used

- **GitHub –** To store source code and run GitHub Actions.
- **Maven –** To build the Java project and create the WAR file.
- **Apache Tomcat –** Application server to run the web app.
- **AWS EC2 –** Cloud VM used to host Tomcat.
- **GitHub Actions –** CI/CD tool used to automate build and deployment.

Introduction: CI/CD Deployment In Tomcat Using Github Actions

Modern software development relies heavily on automation to achieve faster delivery, improved quality, and consistent deployment practices. Continuous Integration and Continuous Deployment (CI/CD) pipelines are essential in enabling teams to build, test, and release applications efficiently without manual intervention. This project focuses on implementing a CI/CD pipeline using **GitHub Actions** to automatically deploy a Java web application to an **Apache Tomcat** server.

The purpose of this work is to design an automated workflow that triggers whenever new code is pushed to the GitHub repository. The pipeline performs several key stages: checking out the source code, setting up the required Java environment, compiling the application using Maven, running tests, packaging the application into a WAR file, and finally deploying it to a Tomcat server. By using GitHub Actions, the entire deployment process becomes repeatable, reliable, and significantly faster compared to manual deployments.

Apache Tomcat is widely used as a lightweight and reliable application server for running Java-based web applications. In this project, Tomcat's **Manager API** is used to deploy the WAR file directly from the CI/CD pipeline. This eliminates the need for developers or administrators to upload the application manually through the Tomcat interface, reducing errors and saving time.

The pipeline also utilizes GitHub Secrets to securely store sensitive information such as server credentials, host details, and deployment URLs. This ensures that authentication details remain protected while still allowing the pipeline to communicate with the Tomcat server during deployment.

Overall, this project demonstrates how automation transforms the software delivery process. By integrating GitHub Actions with Apache Tomcat, the deployment becomes smooth, consistent, and efficient. This CI/CD setup enhances development speed, ensures code stability, and provides a professional, industry-standard workflow suitable for real-world applications.

PROCESS OF DEPLOY WAR FILE IN TOMCAT USING GITHUB ACTIONS

STEP – 1: CREATE EC2 INSTANCES

- Create 4 EC2 Instances for different environments in tomcat.
- Tabel the names for different environments in tomcat (ex: Tomcat-Dev, Tomcat-test, Tomcat-pre-prod, Tomcat-prod)

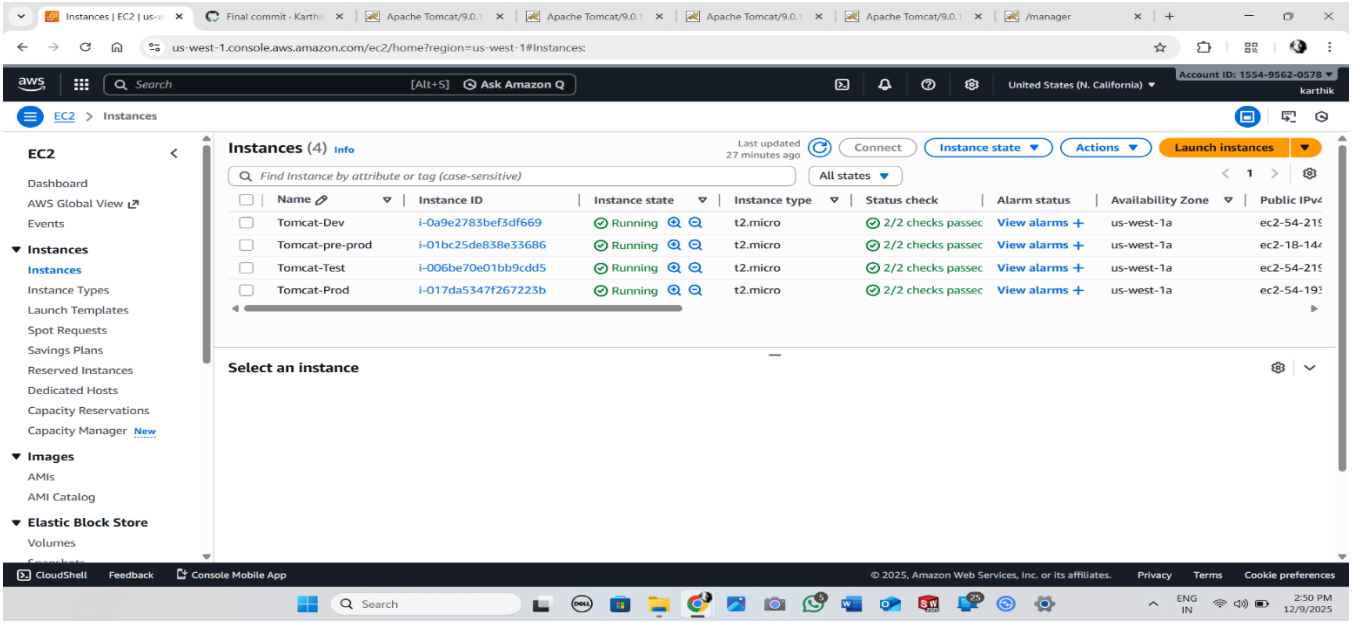


Fig-1: EC2 Instances

STEP – 2: INSTALL JAVA AND MAVEN ON EC2

- Java was installed on the EC2 instance.
- Maven was installed on the EC2 Linux server to build and package the Java application. Maven automates the compilation, testing, and WAR creation steps required before deployment.

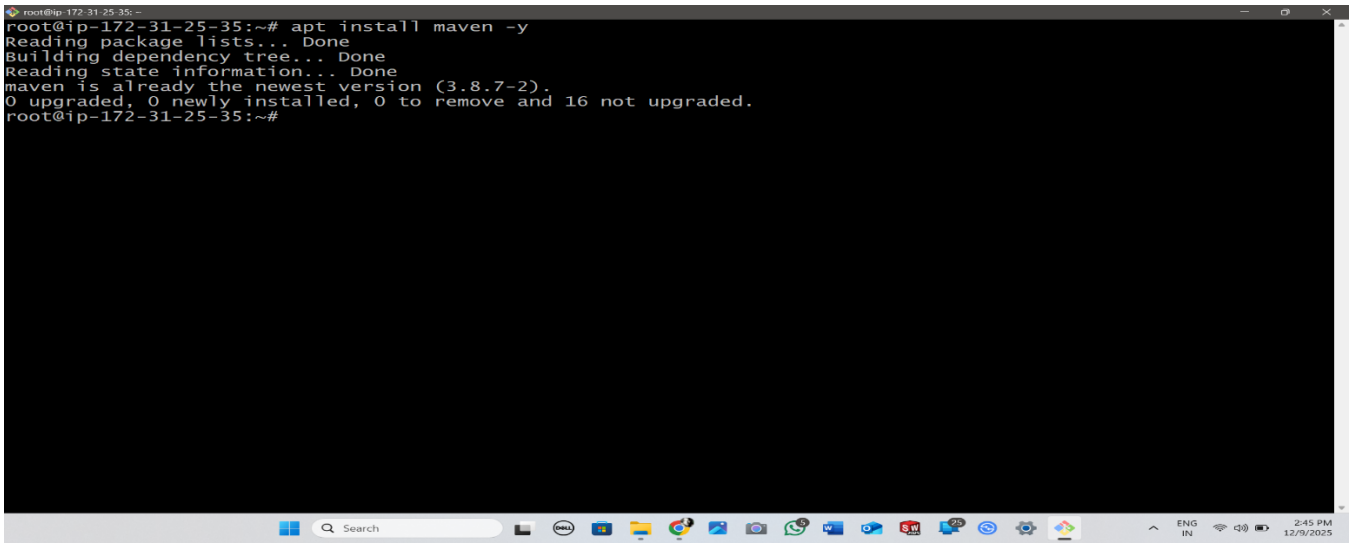


Fig-2: Install Maven On EC2 Instances

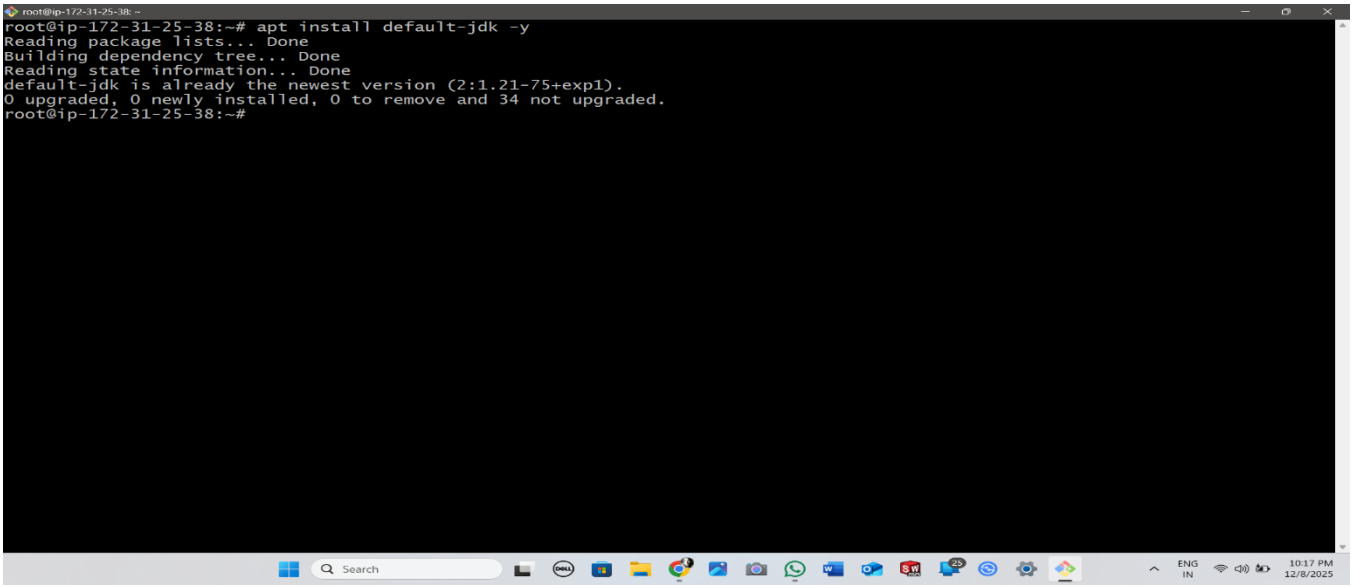


Fig-3: Install Java On EC2 Instances

STEP – 3: INSTALL TOMCAT ON EC2

- Install the tomcat 9 in all EC2 Instances
- Apache Tomcat was downloaded, extracted, and configured to run on port 8080.
- The tomcat-users.xml file was updated to create a user with manager-gui and manager-script roles.

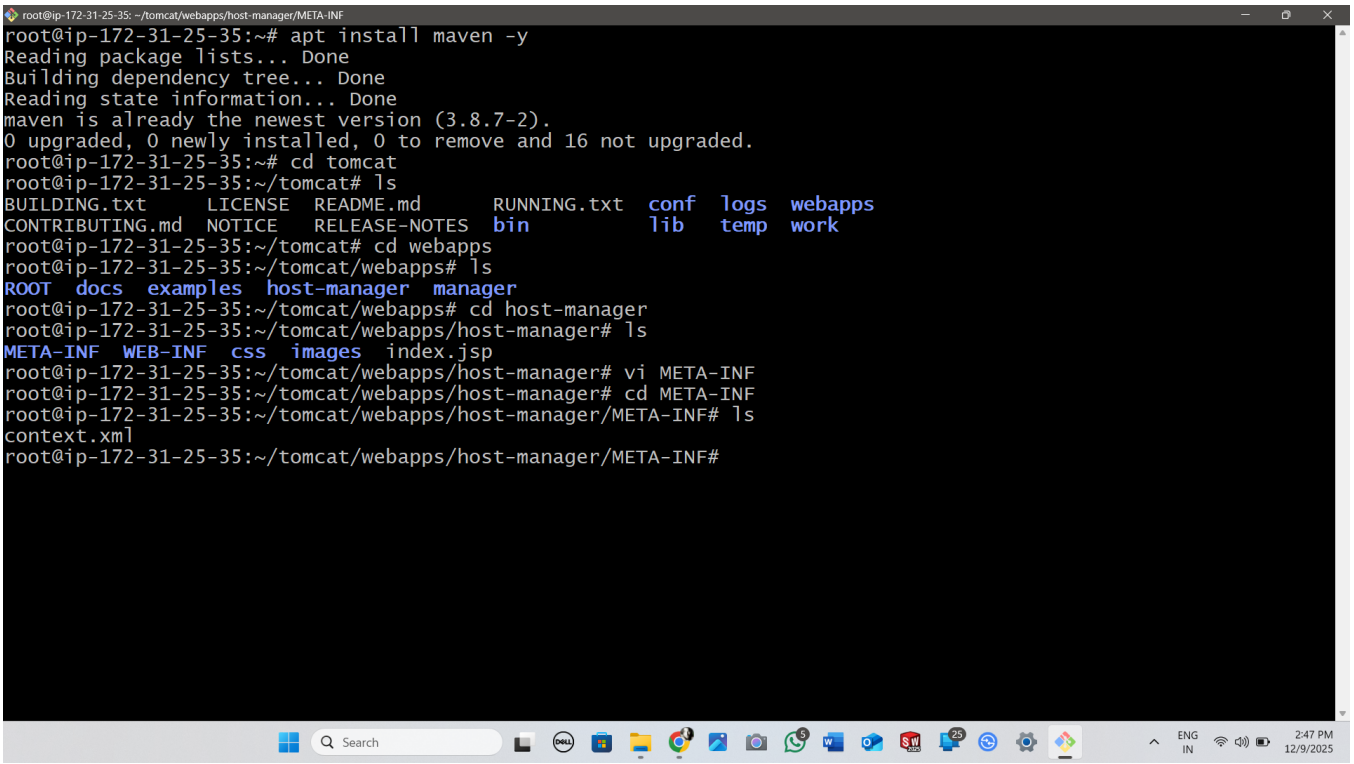


Fig-4: Install Tomcat 9 On EC2 Instances

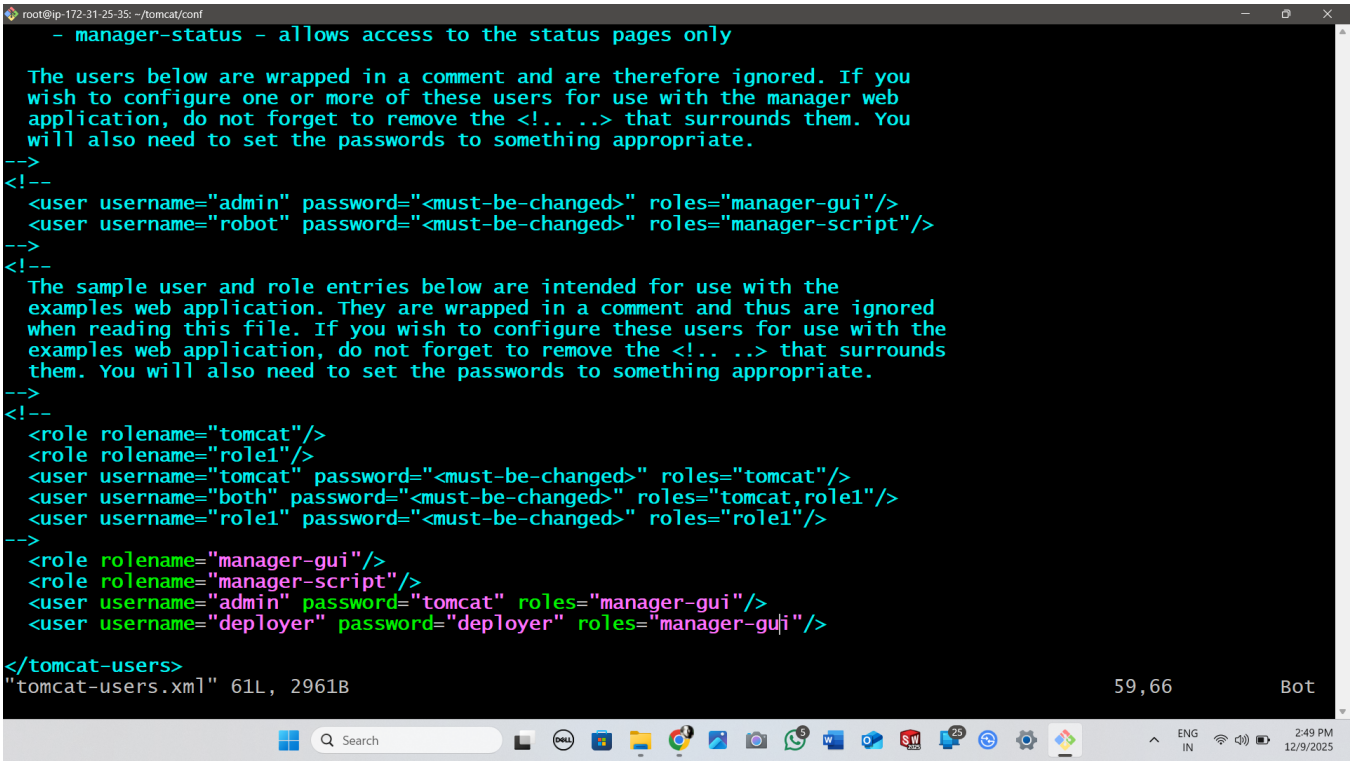


Fig-5: Tomcat-users.xml file updated with manager-gui and manager-script roles.

STEP – 4: CONFIGURE GITHUB REPOSITORY

- Source code was pushed to a GitHub repository.
- Create .github/workflows/main.yaml file was created to define the CI/CD pipeline.
- Add Secrets actions repository for CI/CD pipeline.

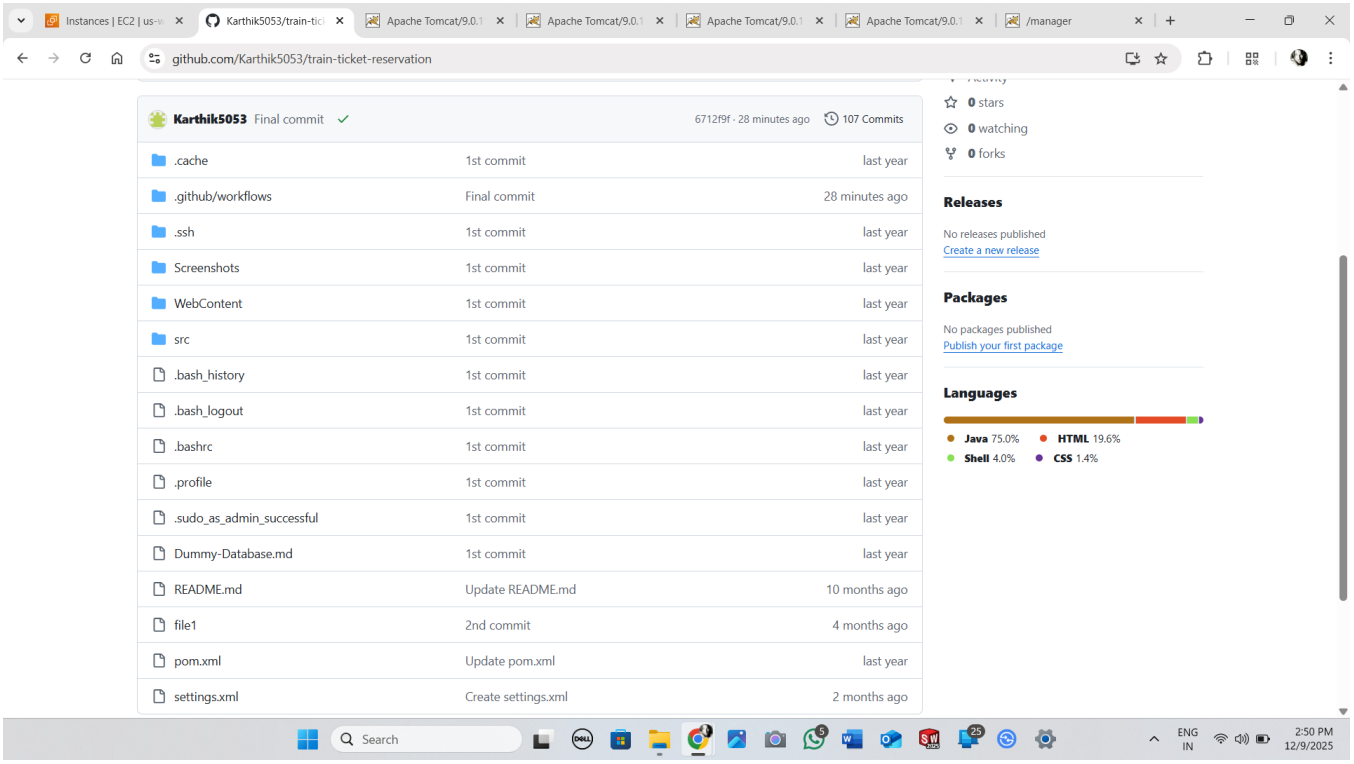


Fig-6: Source code pushed to a GitHub repository.

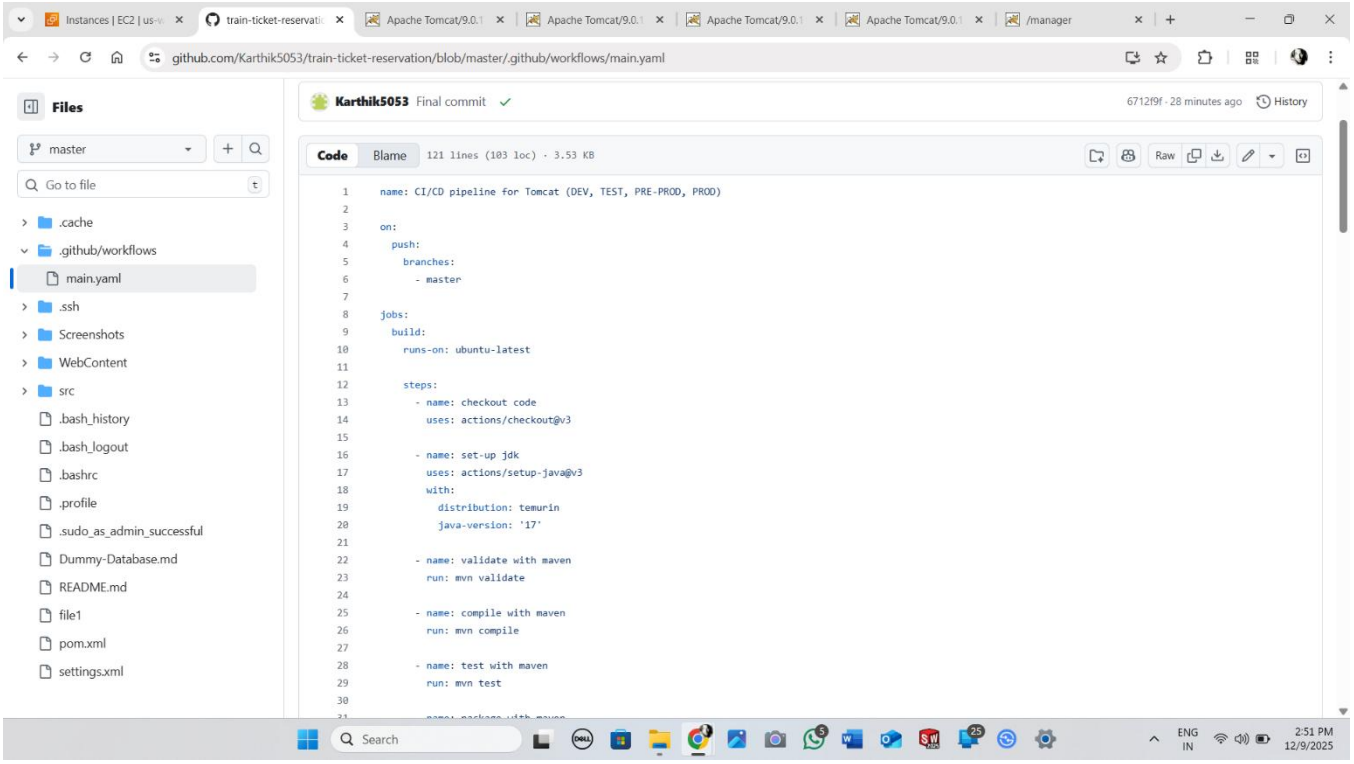


Fig-7: Create .github/workflows/main.yaml file

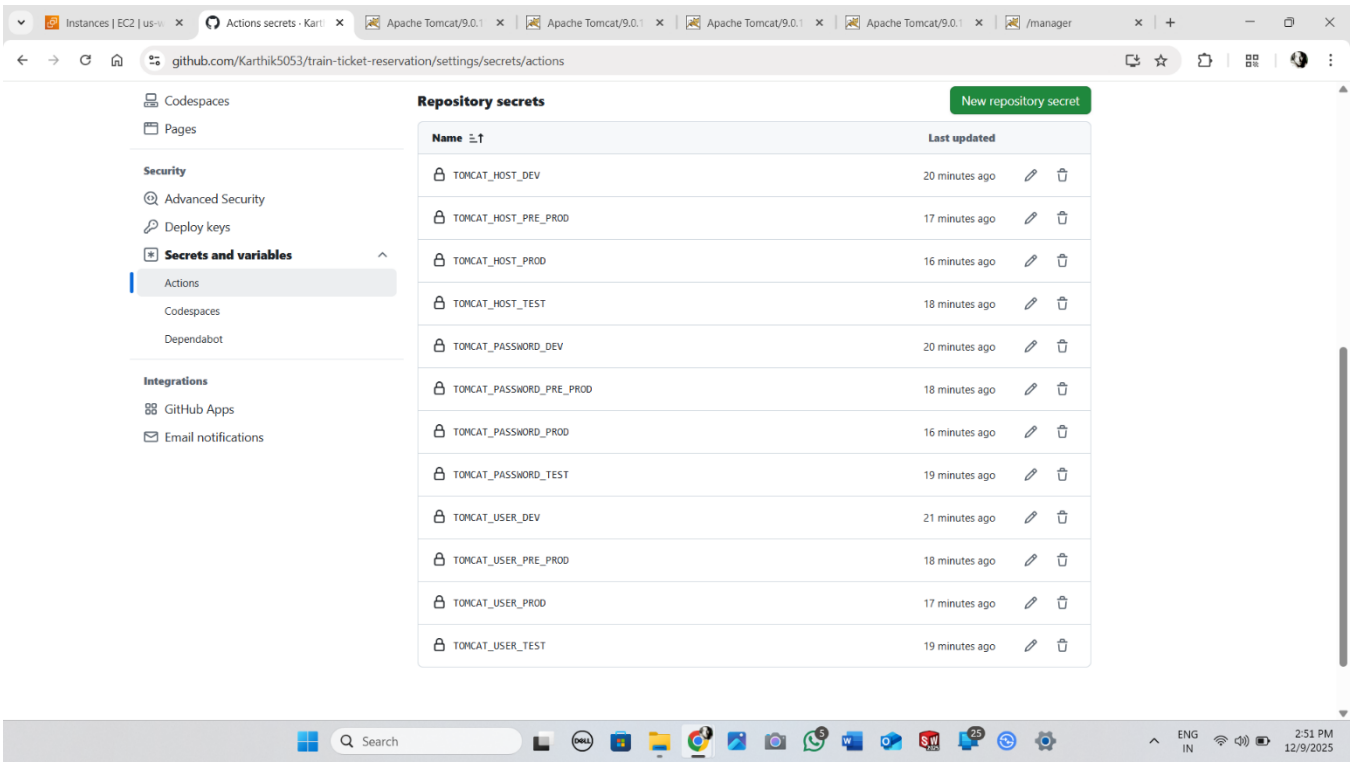


Fig-8: Secrets actions repository

STEP – 5: CI/CD PIPELINE USING GITHUB ACTIONS

- The pipeline checks out the code.
- Sets up Java and runs Maven commands (validate, compile, test, package).
- Uploads the WAR file as an artifact.
- Downloads the WAR in deploy jobs and uses curl to deploy to Tomcat using the Manager API.

STEP – 6: ENVIRONMENT-BASED DEPLOYMENT (DEV/TEST/PRE-PROD/PROD)

- Different Tomcat servers or URLs are used for Dev, Test, Pre-Prod, and Prod.
- GitHub Secrets store different hostnames and credentials for each environment.
- Deploy jobs run in sequence: Dev → Test → Pre-Prod → Prod.

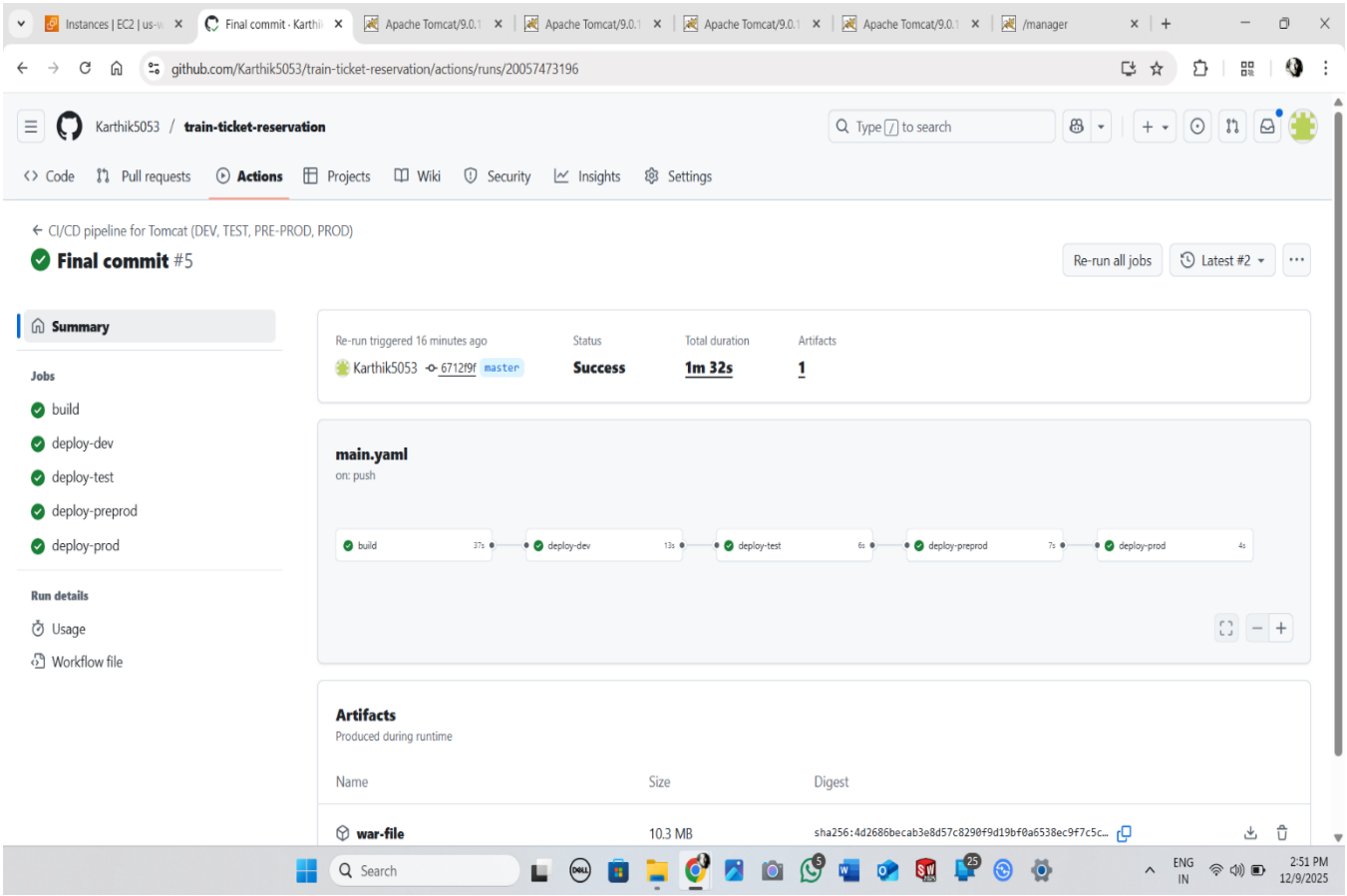


Fig-9: Environment based deployment

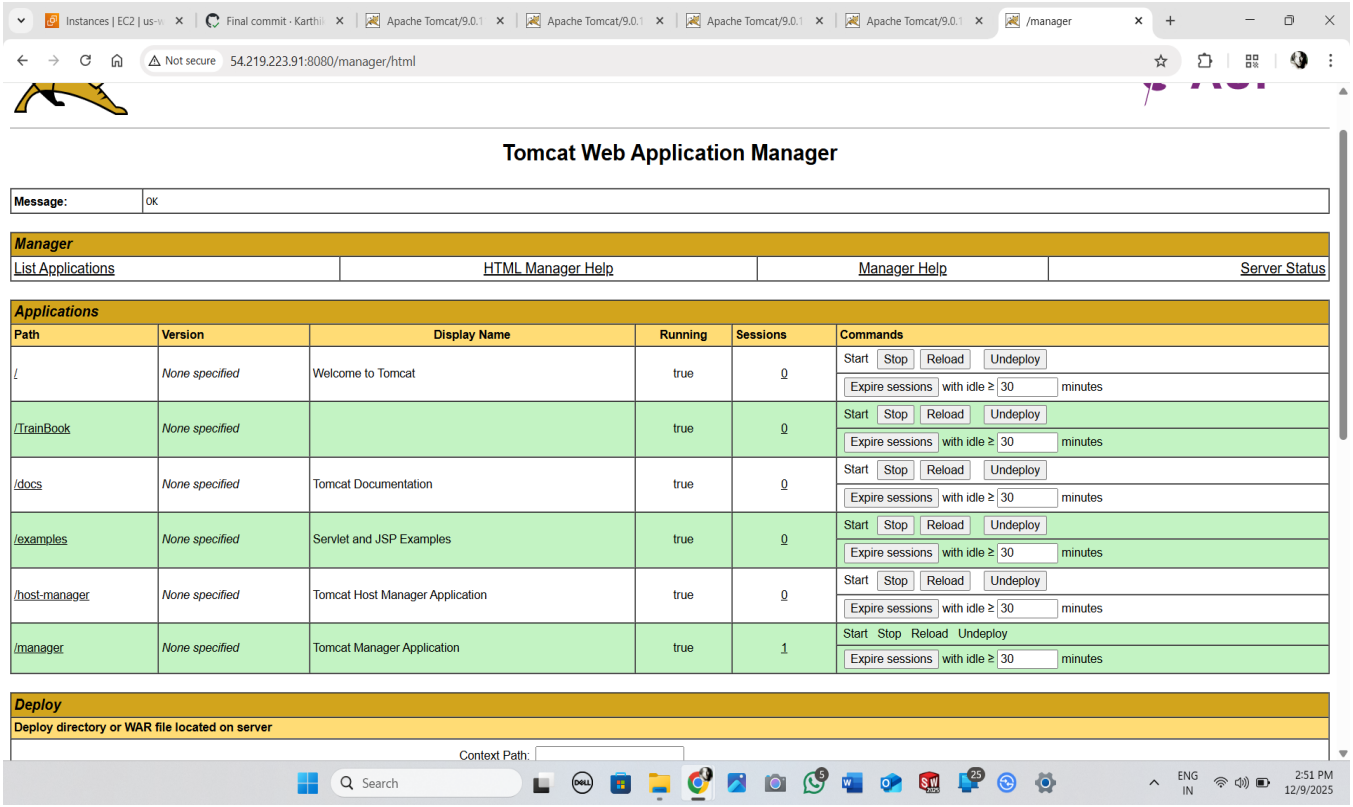


Fig-10: Successful deploy the war file in dev env

STEP – 7: Output

- The application was successfully built and deployed from GitHub Actions to the Tomcat server on EC2.
- After deployment, the application was accessible via the browser using the configured context path (for example, /TrainBook).
- The process removed manual WAR upload and made deployments faster and repeatable.

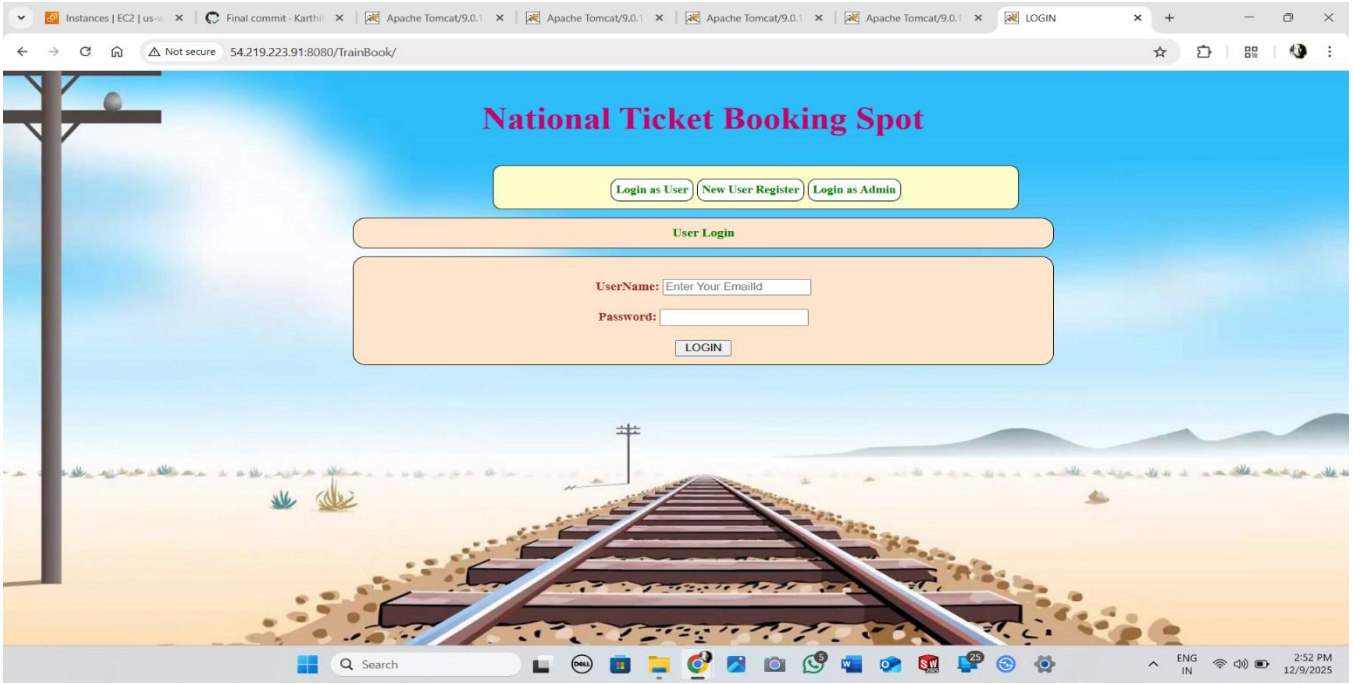


Fig-11: web application

CONCLUSION

This project helped in understanding how a CI/CD pipeline can automate the entire software deployment process. By using GitHub Actions, the build and deployment of a Java web application became faster, easier, and more reliable. The pipeline automatically checked out the code, installed the required tools, ran Maven commands, created the WAR file, and deployed it to the Apache Tomcat server running on an AWS EC2 instance. This removed the need for manual uploading of the application and reduced the chances of human errors.

Creating and configuring the EC2 instance gave practical experience in setting up a cloud server, installing Java, Tomcat, and Maven, and preparing the environment for deployment. Setting up Tomcat users and enabling the Manager API helped in understanding how application servers handle deployments. Using GitHub Secrets ensured that all sensitive information like usernames, passwords, and server IPs stayed secure.

The project also showed how multiple environments such as DEV, TEST, PRE-PROD, and PROD can be handled in a single pipeline. Each environment can receive the same build in sequence, making the deployment process consistent and professional. The use of artifacts made it possible to pass the generated WAR file from the build job to all deployment jobs.

Overall, this project successfully demonstrated the complete DevOps workflow using GitHub Actions, AWS EC2, Maven, and Tomcat. It improved the understanding of automation, CI/CD concepts, environment management, and cloud deployment. The final setup is efficient, reusable, and reflects real-world industry practices in software engineering and DevOps.